

Web Service Design and Practice

07. Cookie, Session, Token

Dr. Kyudong Park

School of Information Convergence

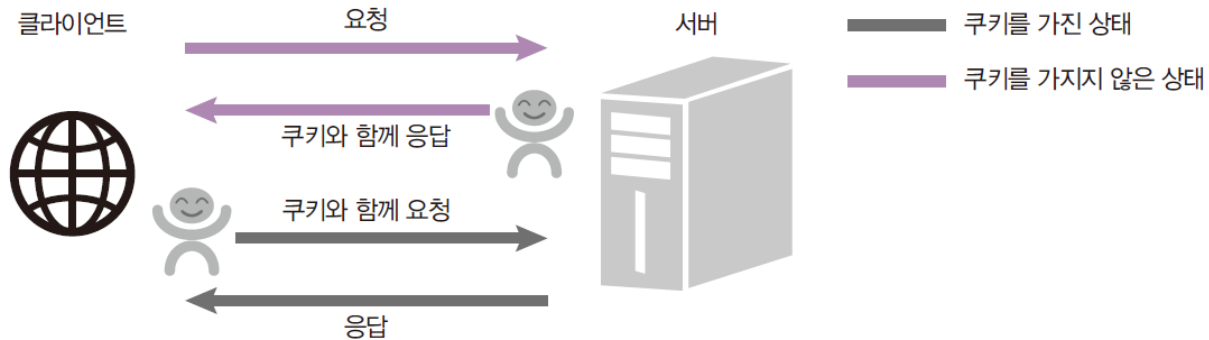


Contents

- [쿠키와 세션](#)
- 암호화와 Token

1. 쿠키의 이해

- 쿠키의 필요성
 - 요청의 단점 : 누가 요청을 보냈는지 모름 (IP주소, 브라우저 정보만 식별 가능)
 - 로그인을 구현하기 위해서 쿠키와 세션이 필요함
- 쿠키는 키-값의 pair
 - 매 요청마다 서버에 동봉해서 보냄
 - 서버는 쿠키를 읽고 누구인지 파악함



1. 쿠키의 이해

- 쿠키 넣는 것을 직접 구현
 - writeHead: 요청 헤더에 입력하는 메서드
 - Set-Cookie: 브라우저에게 쿠키를 설정하라고 명령
- 쿠키: 키=값의 쌍
 - mycookie=test
- localhost:8083에 접속
 - 요청이 전송되고 응답이 왔을 때 쿠키가 설정됨
 - favicon.ico는 브라우저가 자동으로 보내는 요청
 - 두 번째 요청인 favicon.ico에 쿠키가 넣어짐

cookie.js

```
const http = require('http');

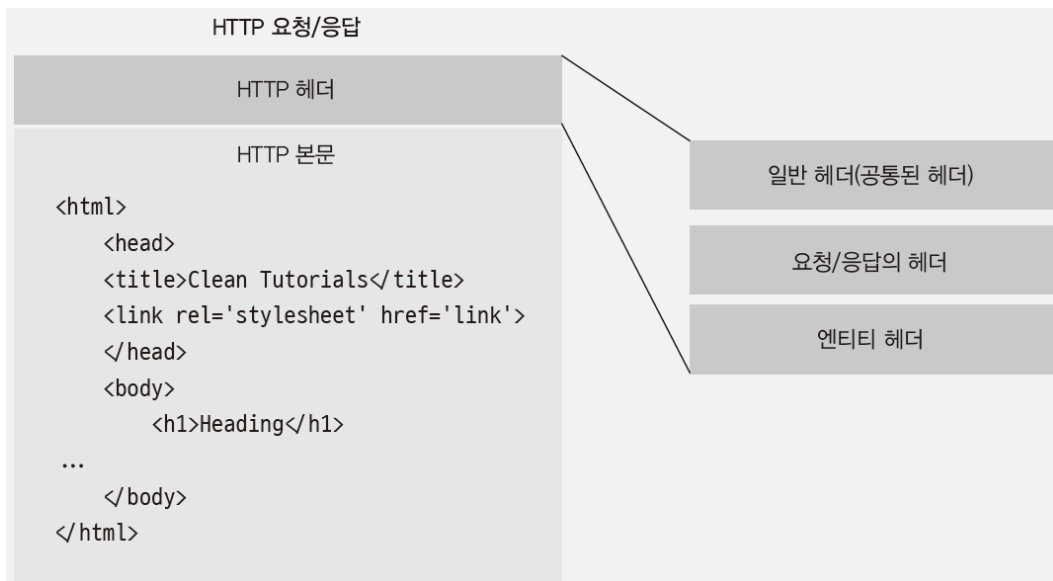
http.createServer((req, res) => {
  console.log(req.url, req.headers.cookie);
  res.writeHead(200, { 'Set-Cookie': 'mycookie=test' });
  res.end('Hello Cookie');
})
.listen(8083, () => {
  console.log('8083번 포트에서 서버 대기 중입니다!');
});
```

콘솔

```
/ undefined
/favicon.ico { mycookie: 'test' }
```

1. 쿠키의 이해

- http 요청과 응답은 헤더와 본문을 가짐
 - 헤더는 요청 또는 응답에 대한 정보를 가짐
 - 본문은 주고받는 실제 데이터
 - 쿠키는 부가적인 정보이므로 헤더에 저장



1. 쿠키의 이해

- writeHead 메서드에 첫 번째 인수로 넣은 값
 - 요청이 성공했는지 실패했는지를 알려줌
 - 2XX: 성공을 알리는 상태 코드입니다. 대표적으로 200(성공), 201(작성됨)이 많이 사용됩니다.
 - 3XX: 리다이렉션(다른 페이지로 이동)을 알리는 상태 코드입니다. 어떤 주소를 입력했는데 다른 주소의 페이지로 넘어갈 때 이 코드가 사용됩니다. 대표적으로 301(영구 이동), 302(임시 이동)가 있습니다.
 - 4XX: 요청 오류를 나타냅니다. 요청 자체에 오류가 있을 때 표시됩니다. 대표적으로 401(권한 없음), 403(금지됨), 404(찾을 수 없음)가 있습니다.
 - 5XX: 서버 오류를 나타냅니다. 요청은 제대로 왔지만 서버에 오류가 생겼을 때 발생합니다. 이 오류가 뜨지 않게 주의해서 프로그래밍해야 합니다. 이 오류를 클라이언트로 res.writeHead로 직접 보내는 경우는 없고, 예기치 못한 에러 발생 시 서버가 알아서 5XX대 코드를 보냅니다. 500(내부 서버 오류), 502(불량 게이트웨이), 503(서비스를 사용할 수 없음)이 자주 사용됩니다.

1. 쿠키의 이해

- <https://github.com/ZeroCho/nodejs-book/tree/master/ch4/4.3>
- 쿠키로 나를 식별하기 (cookie2.js)
 - parseCookies: 쿠키 문자열을 객체로 변환
 - 주소가 /login인 경우와 /인 경우로 나뉨
 - /login인 경우 쿼리스트링으로 온 이름을 쿠키로 저장
 - 그 외의 경우 쿠키가 있는지 없는지 판단
 - 있으면 환영 인사
 - 없으면 로그인 페이지로 리다이렉트

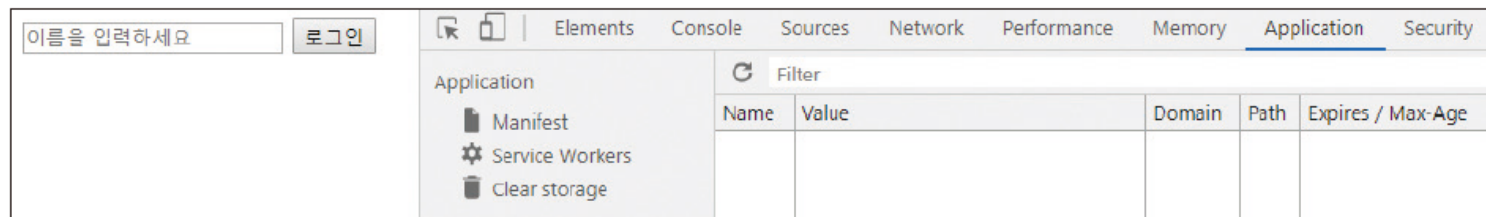
1. 쿠키의 이해

- 쿠키 옵션, Set-Cookie 시 다양한 옵션이 있음
 - **쿠키명=쿠키값**: 기본적인 쿠키의 값입니다. mycookie=test 같이 pair 로 설정합니다.
 - **Expires=날짜**: 만료 기한입니다. 이 기한이 지나면 쿠키가 제거됩니다. 기본값은 클라이언트가 종료될 때까지입니다.
 - **Max-age=초**: Expires와 비슷하지만 날짜 대신 초를 입력할 수 있습니다. 해당 초가 지나면 쿠키가 제거됩니다. Expires보다 우선합니다.
 - **Domain=도메인명**: 쿠키가 전송될 도메인을 특정할 수 있습니다. 기본값은 현재 도메인입니다.
 - **Path=URL**: 쿠키가 전송될 URL을 특정할 수 있습니다. 기본값은 '/'이고 이 경우 모든 URL에서 쿠키를 전송할 수 있습니다.
 - **Secure**: HTTPS일 경우에만 쿠키가 전송됩니다.
 - **HttpOnly**: 설정 시 자바스크립트에서 쿠키에 접근할 수 없습니다. 쿠키 조작을 방지하기 위해 설정하는 것이 좋습니다.

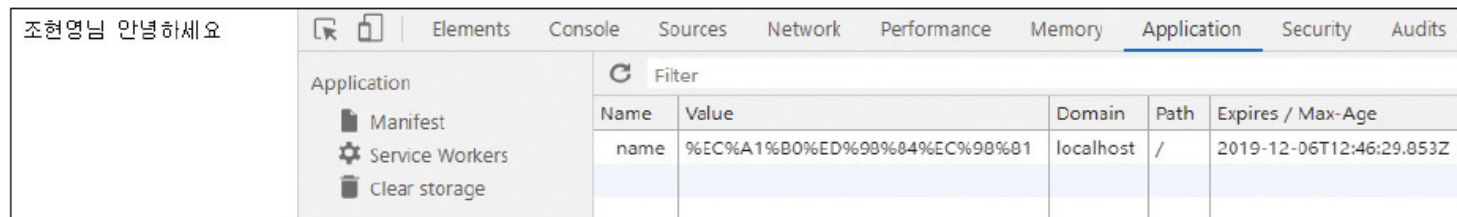
1. 쿠키의 이해

- cookie2.js 의 실행
- 로그인을 하면 쿠키가 생성됨

▼ 그림 4-15 로그인 이전



▼ 그림 4-16 로그인 이후



2. 세션의 이해

- 쿠키의 정보는 노출되고 수정되는 위험이 있음
 - 중요한 정보는 서버에서 관리하고 클라이언트에는 세션 키만 제공
 - 서버에 세션 객체(session) 생성 후, uniqueInt(키)를 만들어 속성명으로 사용
 - 속성 값에 정보 저장하고 uniqueInt를 클라이언트에 보냄
- 세션의 동작 방식
 - 클라이언트가 서버에 접속 시 세션 ID를 발급 받음
 - 클라이언트는 세션 ID에 대해 쿠키를 사용해서 저장하고 가지고 있음
 - 클라이언트는 서버에 요청할 때, 이 쿠키의 세션 ID를 같이 서버에 전달해서 요청
 - 서버는 세션 ID를 전달 받아서 별다른 작업없이 세션 ID로 세션에 있는 클라이언트 정보를 가져와서 사용
 - 클라이언트 정보를 가지고 서버 요청을 처리하여 클라이언트에게 응답

2. 세션의 이해

- <https://github.com/ZeroCho/nodejs-book/tree/master/ch4/4.3>

- session.js

- 세션 서버의 실행

♥ 그림 4-17 로그인 이후

조현영님 안녕하세요		Elements Console Sources Network Performance Memory Application Security				
Application - Manifest - Service Workers - Clear storage		Filter				
		Name	Value	Domain	Path	Expires / Max-Age
		session	1575636661540	localhost	/	2019-12-06T12:56:01.547Z

- 실제로는 세션을 직접 구현하지 말기
 - 제공하는 안전한 library 활용 : express-session

쿠키와 세션의 비교

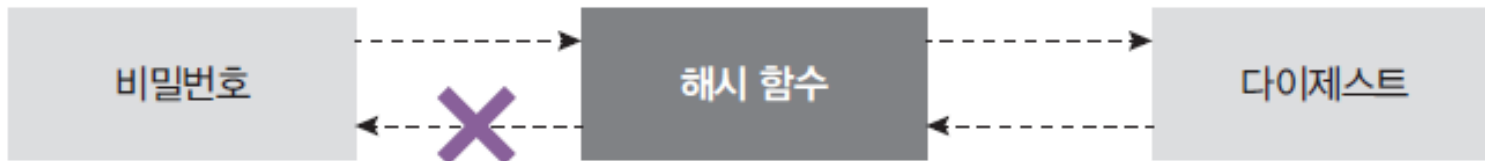
- 쿠키는 서버의 자원을 전혀 사용하지 않으며, 세션은 서버의 자원을 사용
- 보안 면에서 세션이 더 우수
 - 쿠키도 만료시간이 있지만 파일로 저장되기 때문에 브라우저를 종료해도 계속해서 정보가 남아 있을 수 있음 반면 세션은 브라우저가 종료되면 만료시간에 상관없이 삭제됨
 - 쿠키는 클라이언트 로컬에 저장되기 때문에 변질되거나 request에서 스니핑 당할 우려가 있어서 보안에 취약하지만 세션은 쿠키를 이용해서 sessionid 만 저장하고 그것으로 구분해서 서버에서 처리
- 요청 속도는 쿠키가 세션보다 더 빠름
 - 세션은 서버의 처리가 필요하기 때문

Contents

- 쿠키와 세션
- [암호화와 Token](#)

1. 암호화

- 단방향 암호화
 - 암호화는 가능하지만 복호화는 불가능
 - 암호화 : 평문을 암호로 만들
 - 복호화 : 암호를 평문으로 만들
 - 복호화할 필요가 없는, 복호화해서는 안되는 정보에 활용 : 사용자 비밀번호
 - 주로 Hash 기법을 활용



1. 암호화

- Hash 사용하기 (sha512)
 - createHash(알고리즘): 사용할 해시 알고리즘을 넣어줍니다.
 - md5, sha1, sha256, sha512 등이 가능하지만, md5와 sha1은 이미 취약점이 발견
 - 현재는 sha512 정도로 충분하지만, 나중에 sha512마저도 취약해지면 더 강화된 알고리즘으로 변경해야함
 - update(문자열): 변환할 문자열을 넣어줍니다.
 - digest(인코딩): 인코딩할 알고리즘을 넣어줍니다.
 - base64, hex, latin1이 주로 사용되는데, 그중 base64가 결과 문자열이 가장 짧아서 자주 활용됨

```
1  const crypto = require("crypto");
2
3  console.log(
4    "bash64:",
5    crypto.createHash("sha512").update("password1234").digest("base64")
6  );
```

1. 암호화

- 반복적 Hash 사용하기 (pbkdf2)
 - 실제로 많이 활용되는 방식
 - salt 값과 반복횟수를 설정
 - salt 값을 따로 저장해두어야 암호화된 메시지의 동일성 여부 판단 가능



1. 암호화

- 반복적 Hash 사용하기 (pbkdf2 구현)
 - 예제에서는 랜덤한 salt 값 활용, 실제로는 salt 값 고정 필요함

pbkdf2.js

```
const crypto = require('crypto');

crypto.randomBytes(64, (err, buf) => {
  const salt = buf.toString('base64');
  console.log('salt:', salt);
  crypto.pbkdf2('비밀번호', salt, 100000, 64, 'sha512', (err, key) => {
    console.log('password:', key.toString('base64'));
  });
});
```

콘솔

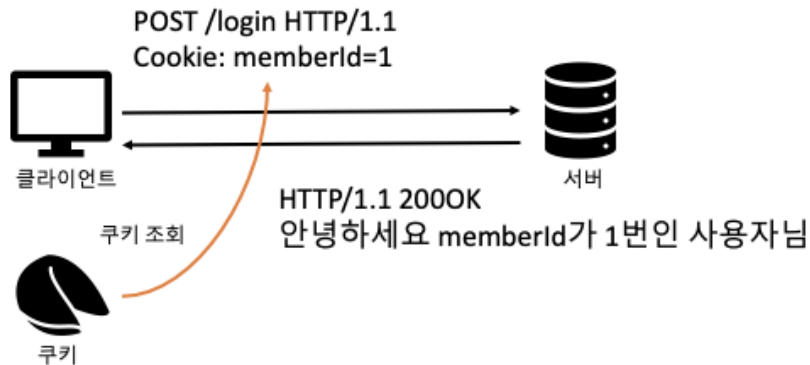
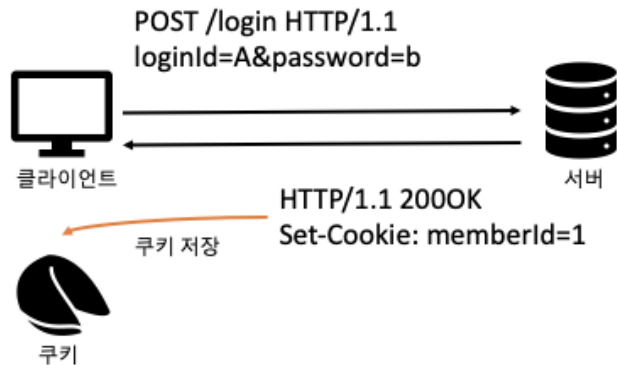
```
$ node pbkdf2
salt: OnesIj8wznyKgHva1fmulYAgjf/0GLmJnwfy8pIABchHZF/Wn2AM2Cn/9170Y1AdehmJ0E5CzLZULps+da
F6rA==
password: b4/FpSrZulVY28trzNXsl4vVfh0KBPxyVAvwnUCWvF1nnXS1zsU1Paq2p68VwUfhB0LDD44hJ0f+tL
e3HMLVmQ==
```

2. JWT

- JWT (JSON Web Token)
 - JSON 형식의 데이터를 저장하는 토큰
 - API 요청을 인증하는 수단으로 활용
 - 쿠키, 세션의 단점을 보완
 - 쿠키 : 암호화 안됨, 사이즈 제한 (4KB), 브라우저마다 형태 다름
 - 세션 : 서버 자원을 사용
 - 토큰 변조가 불가능하고, 내용에 포함해서 전송하므로 서버에서 DB 조회를 하지 않을 수 있음

2. JWT

- 쿠키를 활용한 로그인 사례

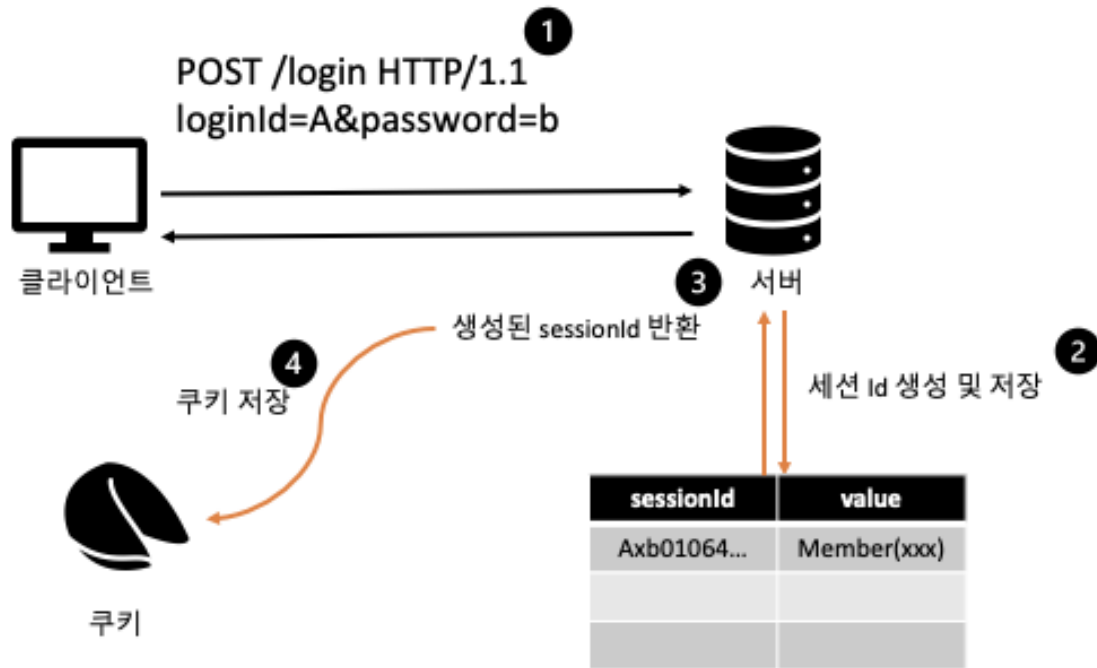


- memberId 가 1이라는 사실이 노출됨
- 쿠키값을 변조할 수 있음

2. JWT

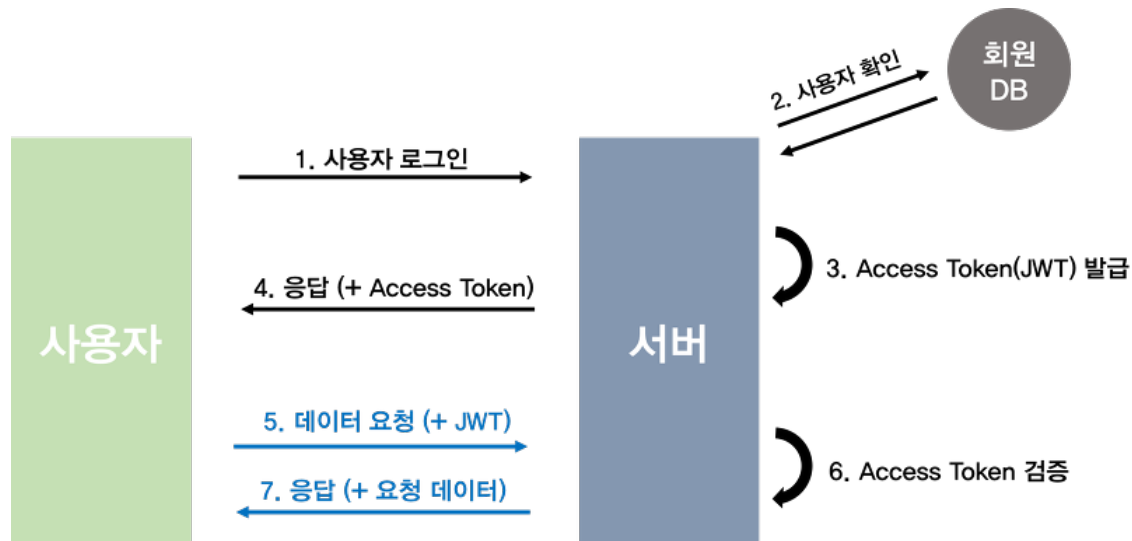
- 세션을 활용한 로그인 사례

- 알 수 없는 정보로 통신함
- 서버는 매번 DB 호출을 해야함



2. JWT

- JWT (JSON Web Token) 를 활용한 로그인 사례



- 기존 쿠키에 검증 절차를 더한 것

2. JWT

- JWT 의 구성
 - header: 토큰 종류와 해시 알고리즘 정보가 들어있음
 - payload: 토큰의 내용물이 인코딩된 부분
 - signature: 일련의 문자열로, 이를 통해 토큰이 변조되었는지 여부 확인
 - JWT 비밀키로 만들어지고, 비밀키가 노출되면 토큰 위조 가능
 - 비밀키는 별도 개발환경 안에서 관리해야함 (공개된 곳에 올라가지 않도록 주의)

2. JWT

- JWT sample
 - <https://jwt.io/>

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.he0ErCNloe4J7Id0Ry2SEDg091KkZkfsRiGsdX_vgEg
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  
) ☐ secret base64 encoded
```

2. JWT

- jwt 관련 모듈 설치
 - `$ npm install jsonwebtoken dotenv`
- 토큰 발급
 - `.sign( {넣을정보}, 비밀키, {만료시간} )`

```
const jwt = require("jsonwebtoken");
const token = jwt.sign({ id: "kdkd" }, "shhhhh", {
  expiresIn: "1m",
  issuer: "kdpark",
});
console.log(token);
```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Imtka2QiLCJpYXQiOiJlOTg0NTY2MDIsImV4cCI6MTY5ODg1NjY2MiwiiaXNzIjoia2RwYXJrIn0. krXIDSEJaOEcvCtXGXGJXb-PZMxDCqq9Jlt4EGeytA

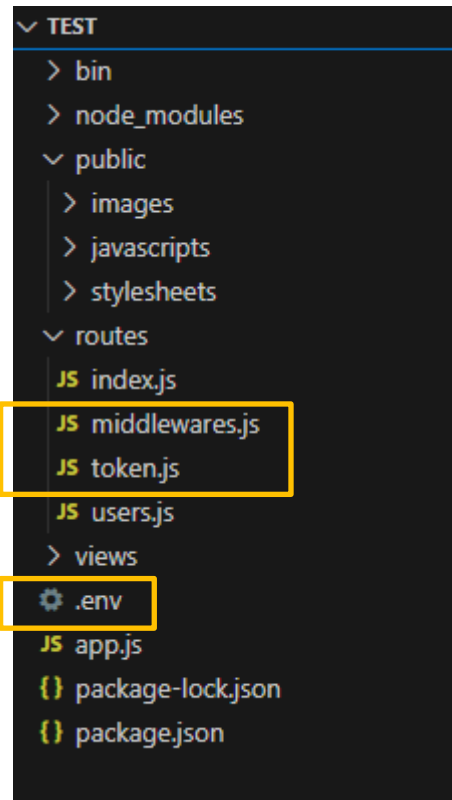
2. JWT

- 토큰 검증
 - 프론트에서는 header 에 넣어서 호출
 - header 와 비밀키를 넣어서 검증
 - 비밀키는 dotenv 를 통해 관리
 - process.env
 - middleware 생성
 - 오류 발생 시, 그에 대한 설명을 return
 - 오류 없으면, 실제 사용자의 호출이 맞다고 가정 (변조 없음)

```
const jwt = require('jsonwebtoken');
...
exports.verifyToken = (req, res, next) => {
  try {
    req.decoded = jwt.verify(req.headers.authorization, process.env.JWT_SECRET);
    return next();
  } catch (error) {
    if (error.name === 'TokenExpiredError') { // 유효 기간 초과
      return res.status(419).json({
        code: 419,
        message: '토큰이 만료되었습니다',
      });
    }
    return res.status(401).json({
      code: 401,
      message: '유효하지 않은 토큰입니다',
    });
  }
};
```

실습

- /token
 - post 요청 시, 토큰 발급
- /token/test
 - get 요청 시, 발급된 토큰이 유효한지 검증
- 테스트는 postman 으로 실시



실습

- middlewares.js

```
const jwt = require("jsonwebtoken");

exports.verifyToken = (req, res, next) => {
  // 인증 완료
  try {
    req.decoded = jwt.verify(req.headers.authorization,
    process.env.JWT_SECRET);
    return next();
  } catch (error) {
    // 인증 실패
    if (error.name === "TokenExpiredError") {
      return res.status(419).json({
        code: 419,
        message: "토큰이 만료되었습니다.",
      });
    }
    return res.status(401).json({
      code: 401,
      message: "유효하지 않은 토큰입니다.",
    });
  }
};
```

실습

- token.js

```
const express = require("express");
const jwt = require("jsonwebtoken");
require("dotenv").config();

const { verifyToken } = require("../middlewares");

const router = express.Router();

router.post("/", async (req, res) => {
  try {
    const id = "myid";
    const nick = "kdpark";
    const token = jwt.sign(
      {
        id,
        nick,
      },
      process.env.JWT_SECRET,
      {
        expiresIn: "1m",
        issuer: "hcclab",
      }
    );
  }
});
```

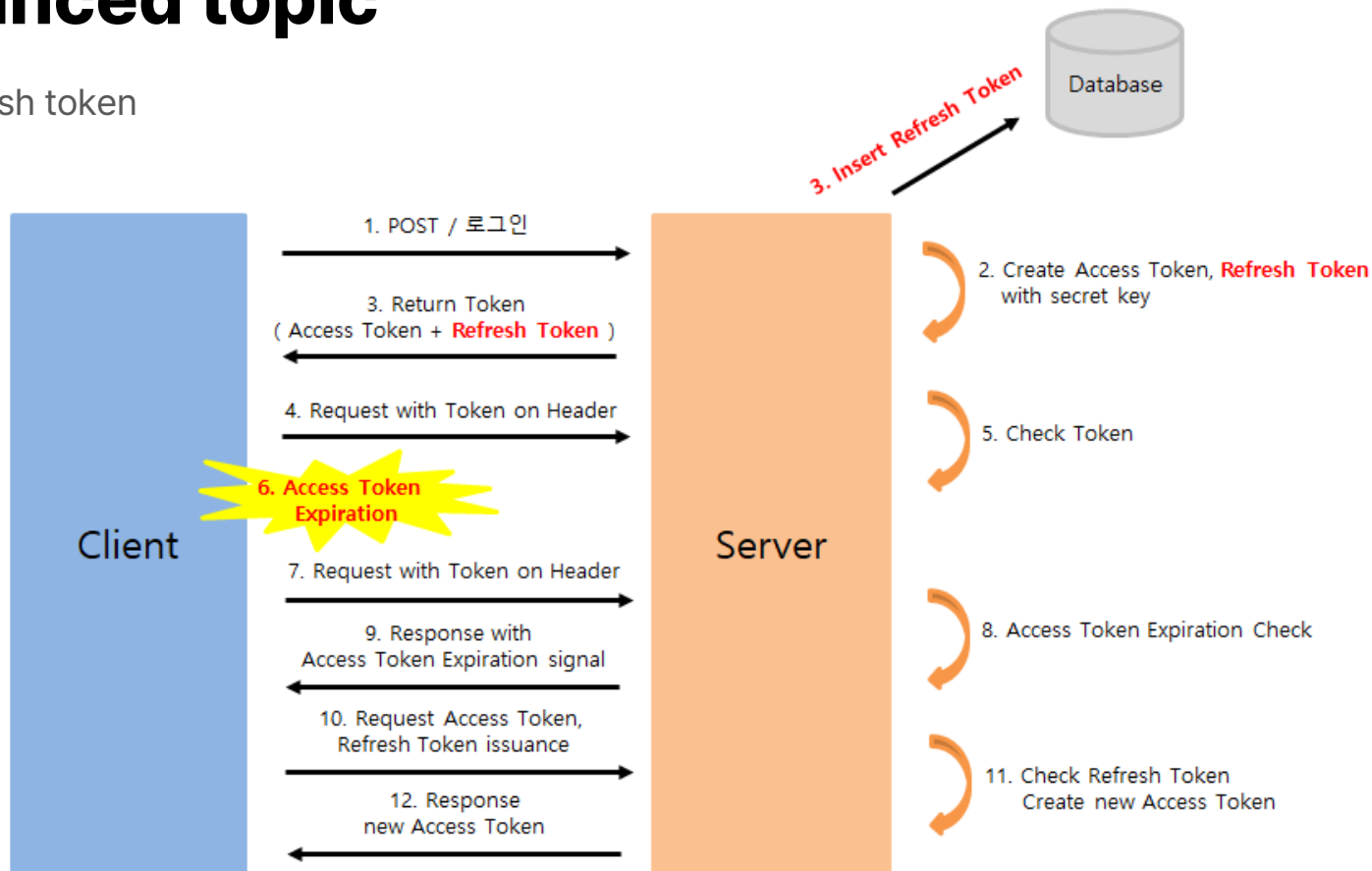
```
return res.json({
  code: 200,
  message: "토큰이 발급되었습니다.",
  token,
});
} catch (error) {
  console.error(error);
  return res.status(500).json({
    code: 500,
    message: "서버 에러",
  });
}
});

router.get("/test", verifyToken, (req, res) => {
  res.json(req.decoded);
});

module.exports = router;
```

Advanced topic

- Refresh token



Summary

- 사용자의 상태 정보를 어떻게 관리할 것인가
 - 사용자 ID를 그대로 저장
 - 세션키로 발급해서 저장
 - JWT 로 저장
- 어디에 저장할 것인가
 - Local Storage : 5~10MB, 명시적으로 삭제 전까지 저장됨
 - Session Storage : 브라우저 종료 시 날아감
 - Cookie : 4KB 이하, 만료시간 설정 가능
- 어떻게 전송할 것인가
 - Cookie 방식 : 보안이 매우 중요할 때 (Http-only Secure SameSite 옵션)
 - Authorization 방식 (Bearer 토큰)

Thank you

Q & A